# Just How Secure Is a 24 Word BIP39 Bitcoin Seed Phrase?

*Entropy, thermodynamics, quantum asterisks, and the much duller ways a wallet actually gets emptied — plus a hard look at what a six-word passphrase is really worth.*

**Ratty2Tails**

EDITOR

TL;DR

- A 24-word BIP39 phrase carries **256 bits** of entropy — a keyspace so large that brute-forcing it is not merely impractical but thermodynamically absurd, full stop.
- Quantum computing does not meaningfully threaten that entropy. Grover's algorithm only halves the effective bit-strength; the real quantum risk is Shor's algorithm against the elliptic-curve math behind an already-exposed public key, which is a separate problem entirely.
- In the real world, seed phrases are almost never broken by brute force. They are lost, photographed, phished, malware-logged, or handed over willingly to a fake wallet app.
- The optional BIP39 passphrase (the "25th word") adds real protection, but only if it is itself hard to guess.
- A genuinely random six-word passphrase, built from a proper 7,776-word diceware list, carries about **77.5 bits** of entropy — strong, but nowhere near a 24-word seed, and dramatically weaker still if the words were chosen by a human rather than a die.

# Introduction and Scope

Every Bitcoin holder who has set up a wallet has, at some point, stared at twelve or twenty-four words on a card and wondered exactly how much protection they are actually buying. It is a fair question, and a strangely hard one to get a straight answer to. Most explanations either wave vaguely at "military-grade encryption" or drown the reader in exponents with no sense of what those exponents mean. This essay tries to do neither. It sets out, in plain terms, what a 24-word seed phrase actually encodes, what it would genuinely take to break one by force, where quantum computing does and does not change the picture, and — because the theoretical keyspace is almost never where real wallets get emptied — where the actual operational risks lie. It closes with a section prompted by a reader question that comes up often enough to deserve its own treatment: how much security does a much shorter, human-scale six-word passphrase actually buy, and where does that number come from.

The scope here is deliberately narrow. This is not a guide to wallet setup, nor an argument for or against any particular custody arrangement. It is a security-arithmetic essay: what the numbers are, where they come from, and what they do and do not tell you about risk.

# What Two Hundred and Fifty-Six Bits Actually Means

A BIP39 mnemonic is not, strictly, a set of English words chosen for their meaning. It is a human-readable encoding of raw binary entropy. Each word is drawn from a fixed list of 2,048 possibilities, and because 2,048 is exactly $2^{11}$, every word carries precisely eleven bits of information. Twenty-four words therefore carry 264 bits — but eight of those bits are consumed by a checksum derived from the other 256, leaving 256 bits of genuine entropy behind the phrase. A twelve-word phrase, by the same arithmetic, carries 128 bits.

Two hundred and fifty-six bits means the space of possible seeds has $2^{256}$ members. Numbers of that size resist intuition, so it helps to place them next to physical quantities that are themselves already close to incomprehensible. The number of grains of sand on Earth is often cited as an example of a very large number; it isn't close. Neither is the number of stars in the observable universe. Even the number of atoms making up the entire planet falls short by roughly ninety orders of magnitude. The 24-word keyspace sits just below the estimated number of atoms in the observable universe itself.

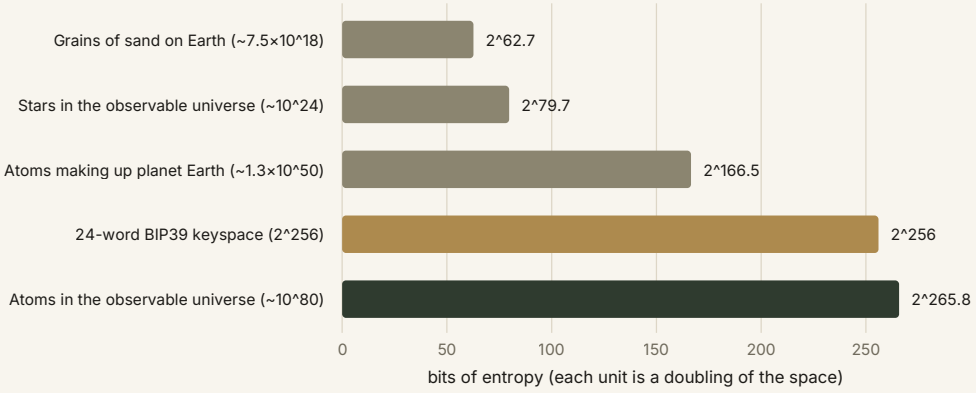


**Fig. 1.** Physical quantities plotted on the same bit-scale as the 24-word BIP39 keyspace. Each gridline is a doubling of the space, so the visual gap between bars understates just how much larger the space really is — $2^{256}$ is not "somewhat bigger" than $2^{167}$, it is roughly $2^{89}$ times bigger, itself a number vastly larger than the count of atoms on Earth.

This is why brute-forcing a 24-word phrase isn't a matter of needing faster computers. It's a matter of physics. Rolf Landauer's 1961 result on the thermodynamics of computation shows that erasing one bit of information has an unavoidable minimum energy cost, set by temperature and Boltzmann's constant.[1] Even a perfectly efficient computer, operating at that theoretical floor and drawing on the entire energy output of a star for its whole lifetime, would still be nowhere near enough to enumerate $2^{256}$ possibilities. This is not a statement

about current technology. It is a statement about what is thermodynamically achievable in this universe, full stop — which is a different, much firmer kind of guarantee than "today's computers aren't fast enough."

## Brute Force, Priced in Joules

It's worth being precise about why "thermodynamically absurd" is not hyperbole. Landauer's principle sets the minimum energy to erase a bit at $kT \ln 2$, where $k$ is Boltzmann's constant and $T$ is the temperature of the computing system. At room temperature this floor is vanishingly small for a single bit — but a brute-force search of $2^{256}$ candidates requires, at an absolute minimum, that many bit-erasure operations. Multiply the per-operation floor by $2^{256}$ and the total energy requirement exceeds, by a wide margin, the entire mass-energy of the observable universe converted with perfect efficiency. Even accepting that real hardware runs many, many orders of magnitude above the Landauer limit — which only makes the practical energy bill worse, not better — the conclusion doesn't change. There is no engineering path from here to there. This is the sense in which a 128-bit space (a twelve-word phrase) is already comfortably unbreakable by exhaustive search for any realistic time horizon, and 256 bits is simply overkill stacked on overkill.

**WORTH REMEMBERING**

None of this means a specific wallet's actual security equals its theoretical bit-strength. The $2^{256}$ figure assumes the entropy was generated honestly and randomly in the first place. A wallet with a flawed, predictable, or deliberately weakened random-number generator can produce a phrase that *looks* like 24 ordinary words while carrying a fraction of the claimed entropy behind it. The math is only as good as the coin flips that fed it.

## The Quantum Asterisk

Quantum computing gets invoked constantly in these discussions, usually with more drama than precision. It's worth separating two genuinely different questions: does quantum computing threaten the entropy of the seed itself, and does it threaten the cryptography built on top of it?

On the first question, the honest answer is: barely. Grover's algorithm, published by Lov Grover in 1996, gives a quadratic speed-up for unstructured search problems — exactly the

kind of problem brute-forcing a seed phrase represents.[2] A quadratic speed-up sounds significant until you do the arithmetic: it effectively halves the bit-strength of the space being searched. A 256-bit space becomes, in effect, a 128-bit problem for a quantum attacker. That is still so far beyond any conceivable computational budget, quantum or otherwise, that it changes nothing about the practical security of a 24-word phrase. Even the more modest 128-bit space of a twelve-word phrase would only fall to a 64-bit-equivalent quantum search — itself still a very large number, and one that assumes a fault-tolerant quantum computer of a scale nobody has built or has any near-term route to building.

The second question is where the real, non-hypothetical quantum risk to Bitcoin actually lives, and it has nothing to do with seed-phrase entropy. Bitcoin's signature scheme relies on the elliptic curve secp256k1, and the hard problem underpinning it — recovering a private key from its corresponding public key — is exactly the kind of structured problem Peter Shor's 1994 algorithm was designed to solve efficiently on a sufficiently large, fault-tolerant quantum computer.[3] Crucially, this attack requires the public key to be exposed on-chain in the first place, which typically happens once an address has been spent from, or if an address has been reused. A seed phrase that has never derived a public, on-chain public key is not vulnerable to this attack path even in a future with practical quantum computers; addresses that have been reused, or whose keys have otherwise leaked their public component, are the actual exposure. This is a live enough concern in the field that it has been the subject of its own dedicated piece.[4]

---

*Grover's algorithm halves the exponent. Shor's algorithm attacks a different problem entirely — and only once a public key has already been exposed.*

---

## Where Seed Phrases Actually Get Broken

Here is the uncomfortable truth for anyone who has spent this essay thinking about entropy and thermodynamics: essentially no real-world Bitcoin theft happens by brute-forcing a seed phrase. The keyspace is not the weak point. People, processes, and hardware are. The realistic attack surface looks like this:

- **Weak or predictable entropy generation.** A wallet with a flawed random-number generator can produce a "24-word phrase" that carries far less real entropy than the wordlist arithmetic implies.

- **Physical loss or theft** of a written, printed, or engraved backup — the single most common cause of permanent, irreversible loss.
- **Digital exposure** — a photo of the words, a note synced to cloud storage, a copy sitting in a password manager or clipboard history.
- **Phishing** — fake wallet software or fraudulent "verification" pages that simply ask the victim to type the phrase in.
- **Malware and keyloggers** present at the moment the phrase is entered.
- **Supply-chain compromise** of hardware wallets, particularly devices bought second-hand or through unofficial channels.
- **Physical degradation** of a paper backup — fire, water, or simple fading over years, which is why steel or titanium backup plates exist.

Every item on that list has nothing to do with how many bits a checksum consumes. This is the central, slightly deflating point of the whole essay: the cryptography is essentially solved. The operational discipline around it is where almost all the real risk still lives.

## The Twenty-Fifth Word

BIP39 supports an optional extra layer: a user-supplied passphrase, often nicknamed the "25th word," which is not part of the wordlist-derived entropy at all. Instead, the mnemonic sentence and this passphrase are run together through PBKDF2 with HMAC-SHA512, using 2,048 iterations, to derive the final 512-bit seed.[5][6] Critically, a different passphrase — including no passphrase at all — produces a completely different, entirely valid-looking wallet from the exact same 24 words. Anyone who finds or steals the written words alone cannot derive the actual wallet without also knowing the passphrase, and the scheme offers a genuine form of plausible deniability: a decoy wallet with a small balance can sit behind the bare words, while the real funds live behind a passphrase only the owner knows.

That protection, however, is only as strong as the passphrase itself is hard to guess — which is precisely the question the rest of this essay turns to.

## How Secure Is a Six-Word Passphrase?

This is the question that prompted the most follow-up mail after this essay first went out in shorter form, so it earns a proper treatment. "Six words" sounds modest next to twenty-four, and the instinct is to assume it's proportionally weaker — a quarter of the phrase, roughly a

quarter of the security. The real answer is both more specific and more interesting than that, and it depends almost entirely on one question: *where did the words come from?*

### The diceware baseline

The rigorous way to build a word-based passphrase is diceware: roll physical dice to select each word from a large, fixed public wordlist, so that every word is genuinely and independently random. The Electronic Frontier Foundation's long wordlist, a widely used standard for this purpose, contains exactly 7,776 words — chosen because it's $6^5$, a number that maps neatly onto five rolls of a six-sided die.[7] Each word drawn this way carries $\log_2(7{,}776)$, or about 12.9 bits, of entropy. Six such words therefore carry roughly **77.5 bits** — a keyspace of a little over $2{\times}10^{23}$ possibilities.

Seventy-seven and a half bits is not weak in any absolute sense. It sits comfortably beyond the reach of any realistic brute-force attack today: even granting an attacker an implausibly generous one trillion guesses per second against a completely unhardened hash, exhausting the space would still take on the order of several thousand years. Fold in the fact that a BIP39 passphrase is run through 2,048 rounds of PBKDF2 rather than a single fast hash, and that timeline stretches very much further. But 77.5 bits is meaningfully smaller than the 128 bits of a twelve-word seed, and dramatically smaller than the 256 bits of a twenty-four-word one — it is, roughly speaking, closer in scale to "a strong password" than to "a Bitcoin seed."

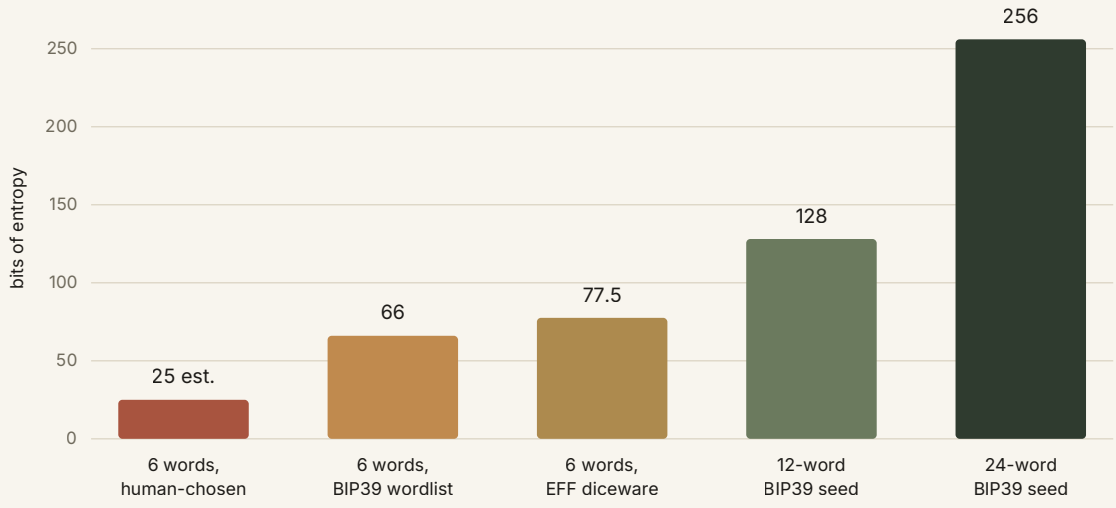


**Fig. 2.** Entropy across passphrase and seed-phrase constructions. The gap between "human-chosen" and "diceware" using the identical word count is the entire point of this section: the wordlist and the randomness source matter far more than the word count on its own.

### The wordlist matters as much as the word count

The 77.5-bit figure assumes the full 7,776-word EFF list. Swap in a smaller list and the number falls fast, because entropy per word is a logarithm of list size, not a linear function of it. Six words drawn randomly from BIP39's own 2,048-word list — sometimes mistakenly reached for as a passphrase source because it's already sitting in the wallet software — carry only 66 bits. Six words from a "memorable words" list curated down to a few thousand common terms for easy recall can fall to somewhere in the region of 55–65 bits, depending on the list. None of these are weak exactly, but each step down erodes the safety margin meaningfully, and margin is precisely what you're paying for with the extra word count in the first place.

### Human choice is the real cliff edge

The much bigger drop happens when a person picks their own six words instead of rolling dice. Decades of password research — most notably Joseph Bonneau's large-scale analysis of real-world password choices — have shown repeatedly that human-generated "random" selections are nothing of the sort.[8] People gravitate toward common words, familiar phrases, names, and predictable grammatical patterns; a self-composed six-word phrase built to feel memorable might carry the theoretical potential of 77 bits on paper, but its *actual*, guessable entropy against an attacker armed with a language model and a dictionary of common phrase patterns is often closer to 20–30 bits — not far removed from a mediocre eight-character password. NIST's digital identity guidelines make essentially the same point in policy form: length alone is a poor proxy for strength once human predictability is in the mix.[9] The gap between "6 words, human-chosen" and "6 words, EFF diceware" in the chart above isn't a rounding error; it's the difference between a passphrase an attacker's dictionary might land on in an afternoon and one that requires a genuinely astronomical search.

Approximate entropy by construction method, six words, for comparison against BIP39 seed lengths

| CONSTRUCTION | ENTROPY | PRACTICAL READ |
|---|---|---|
| Self-composed, memorable phrase | ~20–30 bits (est.) | Within reach of dictionary and pattern-based attacks |
| 6 random words, small curated list | ~55–65 bits | Strong against casual attack, soft against a determined, well-resourced one |
| 6 random words, BIP39's 2,048-word list | 66 bits | Solid, but a poor choice of list for this purpose |
| 6 random words, EFF diceware (7,776 words) | 77.5 bits | Very strong; the recommended baseline |
| 12-word BIP39 seed | 128 bits | Unbreakable by any realistic brute force |
| 24-word BIP39 seed | 256 bits | Thermodynamically unbreakable |

### The practical upshot

If a six-word passphrase is being used as the BIP39 "25th word" layer, the rule of thumb is simple: it is only as good as its worst assumption. Roll real dice against the full EFF long list, don't reuse the passphrase anywhere else, and don't lean on personal meaning to make it memorable — personal significance is exactly what makes a phrase guessable. For anything genuinely high-value, extending to seven or eight diceware words pushes the figure past 90 and then past 100 bits respectively, closing most of the remaining gap with a twelve-word seed for comparatively little extra effort to remember. Six words, done properly, is a strong passphrase. Six words, composed for memorability, is a much weaker one wearing the same costume.

## Summary

A 24-word BIP39 seed phrase encodes 256 bits of entropy, a keyspace so large that brute-forcing it isn't a matter of faster hardware but of physics: the energy required exceeds what the universe has to offer, regardless of how the computation is arranged. Quantum computing doesn't meaningfully change that picture for the entropy itself — Grover's algorithm only halves the effective exponent — though it does pose a genuine long-

horizon risk to exposed public keys via Shor's algorithm, a separate and narrower problem tied to address reuse rather than seed strength.

In practice, the keyspace is essentially never the point of failure. Wallets are compromised through weak randomness, physical loss, digital exposure, phishing, malware, and supply-chain tampering — all operational failures, not cryptographic ones. The optional BIP39 passphrase adds a genuinely useful extra layer, including a form of plausible deniability, but its strength depends entirely on how it was generated. A properly rolled six-word diceware passphrase carries around 77.5 bits — strong, but well short of even a twelve-word seed — while a six-word phrase composed by a human for memorability can carry a small fraction of that in practice. The lesson holds across the whole essay: the math behind Bitcoin's key security is, for all practical purposes, already solved. What remains fragile is everything human beings do around it.

# References


[1] Landauer, R. (1961). *Irreversibility and Heat Generation in the Computing Process.* IBM Journal of Research and Development, 5(3), 183–191.

[2] Grover, L. K. (1996). *A Fast Quantum Mechanical Algorithm for Database Search.* Proceedings of the 28th Annual ACM Symposium on Theory of Computing.

[3] Shor, P. W. (1997). *Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer.* SIAM Journal on Computing, 26(5), 1484–1509.

[4] Ratty2Tails. *When Do We Panic?* The Ratty2Tails Papers, The Machine Series No. I.

[5] Palatinus, M., Rusnak, P., Voisine, A., & Bowe, S. *BIP39: Mnemonic Code for Generating Deterministic Keys.* Bitcoin Improvement Proposals.

[6] Kaliski, B. (2000). *PKCS #5: Password-Based Cryptography Specification, Version 2.0.* RFC 2898, Internet Engineering Task Force.

[7] Electronic Frontier Foundation. *Deep Dive: EFF's New Wordlists for Random Passphrases.*

[8] Bonneau, J. (2012). *The Science of Guessing: Analyzing an Anonymized Corpus of 70 Million Passwords.* IEEE Symposium on Security and Privacy.

[9] National Institute of Standards and Technology. *Digital Identity Guidelines: Authentication and Lifecycle Management.* NIST Special Publication 800-63B.